

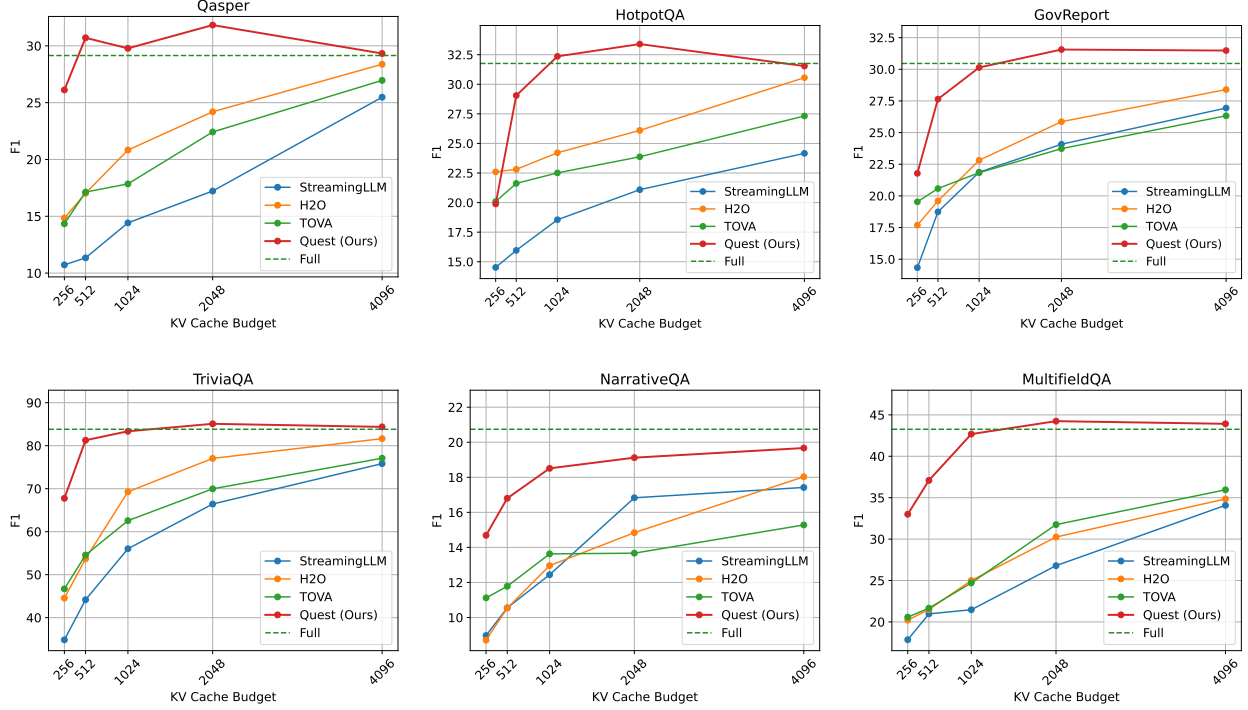


Figure 7. We evaluate Quest and baselines across six long context datasets with various token budgets. Quest constantly surpassing all baselines at all datasets and all token budgets. For most of the dataset, Quest reaches comparable accuracy with a 1K token budget. To evaluate the impact of different methods on the model’s ability to retrieve long-dependency information, we simulate decoding by feeding the task’s question to the model token by token.

under the configuration of Llama2-7B on an RTX4090 with CUDA 12.2 in Sec 4.3.1. Besides, we show the end-to-end speedup of Quest in text generation as shown in Sec 4.3.2. We compare Quest with a normal attention implementation from the original FlashInfer. To demonstrate the improvement, we qualitatively compare efficiency under the same accuracy between Quest and baselines in Sec 4.3.3. Note that we use an Ada 6000 GPU (NVIDIA, 2023) in end-to-end evaluations for longer context length.

4.3.1. KERNEL EVALUATION

Due to the memory-bound nature of LLM inference, the speedup of Quest is proportional to the sparsity ratio (which is equivalent to memory movement reduction). We quantify this effect in Fig. 8, which evaluates per-kernel performance with NVIDIA’s benchmark tool NVBench (NVIDIA, 2024).

Criticality estimation We evaluate the latency of criticality estimation in Quest under different sequence lengths and page sizes. At short sequence length, the memory bandwidth utilization of estimation is smaller than that of FlashInfer, as the total memory load size is not enough to fully utilize GPU memory bandwidth. As sequence length grows, the relative performance improves and approaches 1/Page Size since estimation only consumes one token per page. Note

that techniques like quantization or larger page size can further reduce the additional memory usage.

Top-K filtering We enable the Top-K filtering in Quest with a batched Top-K CUDA operator from a vector search kernel library RAFT (Zhang et al., 2023a). We test the latency of Top-K filtering under different sequence lengths and token budgets. Since Criticality estimation reduces one entire token into one criticality score, Top-K filtering has limited memory movement compared to other operators, thus having a low latency overhead of 5-10 us for sequence length less than 128k.

Approximate attention Since Quest is compatible with PageAttention, approximate attention can be easily implemented by feeding Top-K page indices as sparse loading indices. We compare Quest’s approximate attention with the original attention of FlashInfer under different sequence lengths and token budgets with a 16 page size. At a given token budget B , the latency of Approximate attention is a constant regardless of the sequence length. Since Approximate attention introduces minimal overhead, it has a similar latency as FlashInfer at sequence length B .

We further evaluate Quest’s attention mechanism, which combines Criticality estimation, Top-K filtering, and Ap-

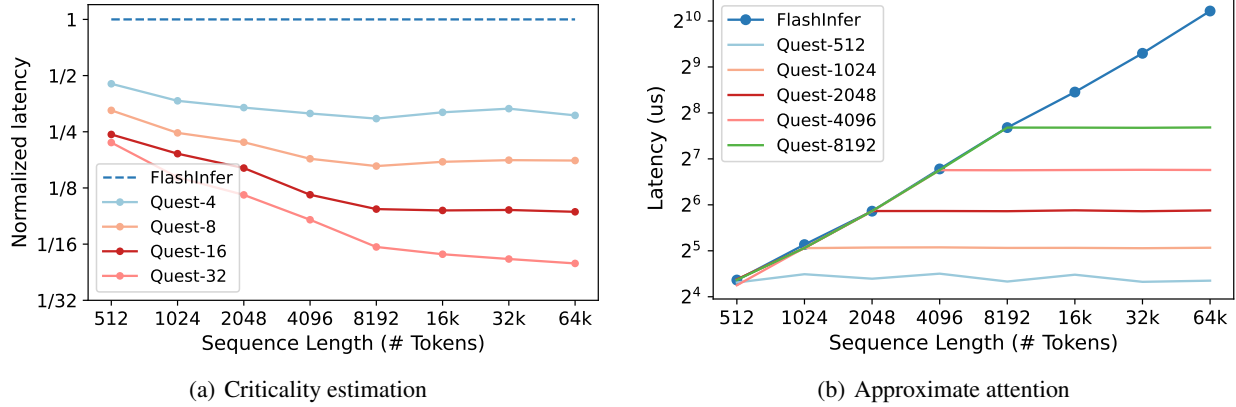


Figure 8. We measure the latency of individual kernels in Quest. (a) As sequence length increases, the relative criticality estimation latency decreases to $1/\text{Page Size}$ of FlashInfer. (b) Approximate attention with token budget K consumes constant time irrelevant to total sequence length and reaches similar performance of FlashInfer at sequence length K .

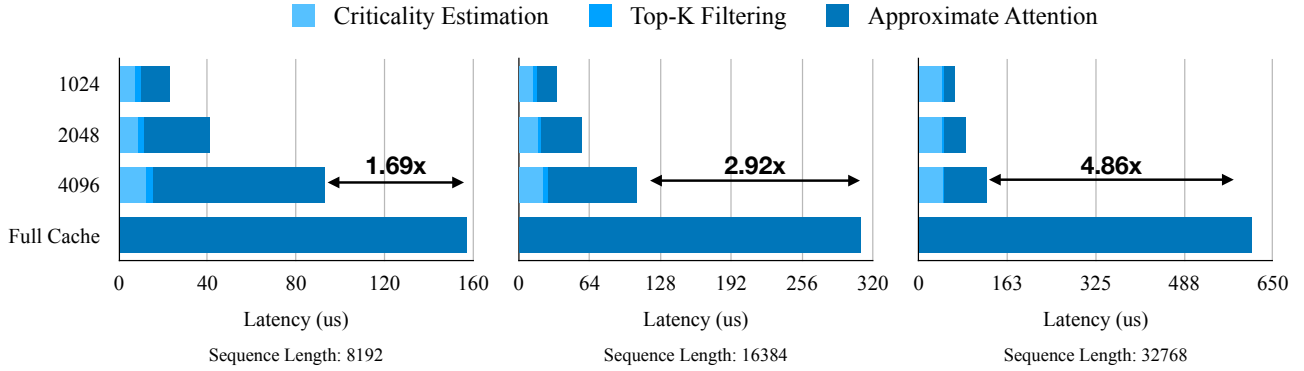


Figure 9. Quest self-attention time breakdown compared to FlashInfer. At all sequence lengths, Quest significantly outperforms FlashInfer, as the memory movement is reduced. At sequence length 32K with token budget 2048, Quest speeds up self-attention by $7.03\times$.

proximate attention, on the Llama2-7B model using the PyTorch profiler. We show the time breakdown of Quest in Fig. 9 on various sequence lengths. Quest reduce the self-attention time by $7.03\times$ compared with FlashInfer at 32K sequence length with 2048 token budget.

4.3.2. END-TO-END EVALUATION

To show the practical speedup of Quest, we deploy the framework into real-world single-batch scenarios. We measure the average latency of generating one token in the decode stage under different sequence lengths and token budgets. Note that we do not measure the sampling process since its execution time is smaller and depends on the setting. We compare Quest with a full KV cache baseline which is implemented by FlashInfer. As shown in Fig. 10, Quest outperforms FlashInfer at all sequence lengths. The latency of Quest grows significantly slower than FlashInfer when the sequence length increases, as Quest maintains

similar token budgets. At sequence length 32K and token budget 2048, Quest boosts inference speed by $1.74\times$ with FP16 weights and $2.23\times$ with 4-bit quantized weight.

4.3.3. COMPARISON WITH BASELINES

To demonstrate the performance improvements of Quest, we compare the inference efficiency of different attention mechanisms under the same accuracy constraint, i.e. lossless accuracy of six tasks from LongBench. We show token budgets needed for the lossless accuracy target by different attention mechanisms in Fig 11(a). For example, NarrativeQA exhibits an average context length of 24K tokens. To achieve lossless accuracy, TOVA requires a token budget of 14K, whereas Quest necessitates only 5K tokens leading to much higher sparsity.

However, none of the baselines included a kernel implementation of their proposed method. Consequently, we conduct a qualitative analysis of the baselines' self-attention

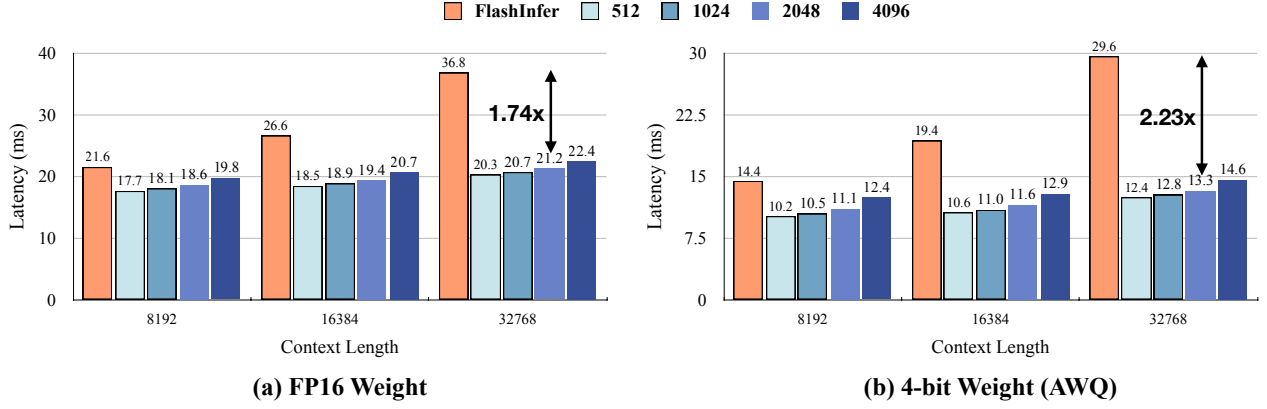


Figure 10. End-to-end latency of Quest. For all sequence lengths, Quest significantly outperforms FlashInfer. Increasing the sequence lengths only slightly changes the latency of Quest. At a given sequence length, Quest’s latency slightly increases as the token budget grows. With sequence length 32K, token budget 2048, 4-bit weight quantization, Quest speedup end-to-end inference by $2.23\times$.

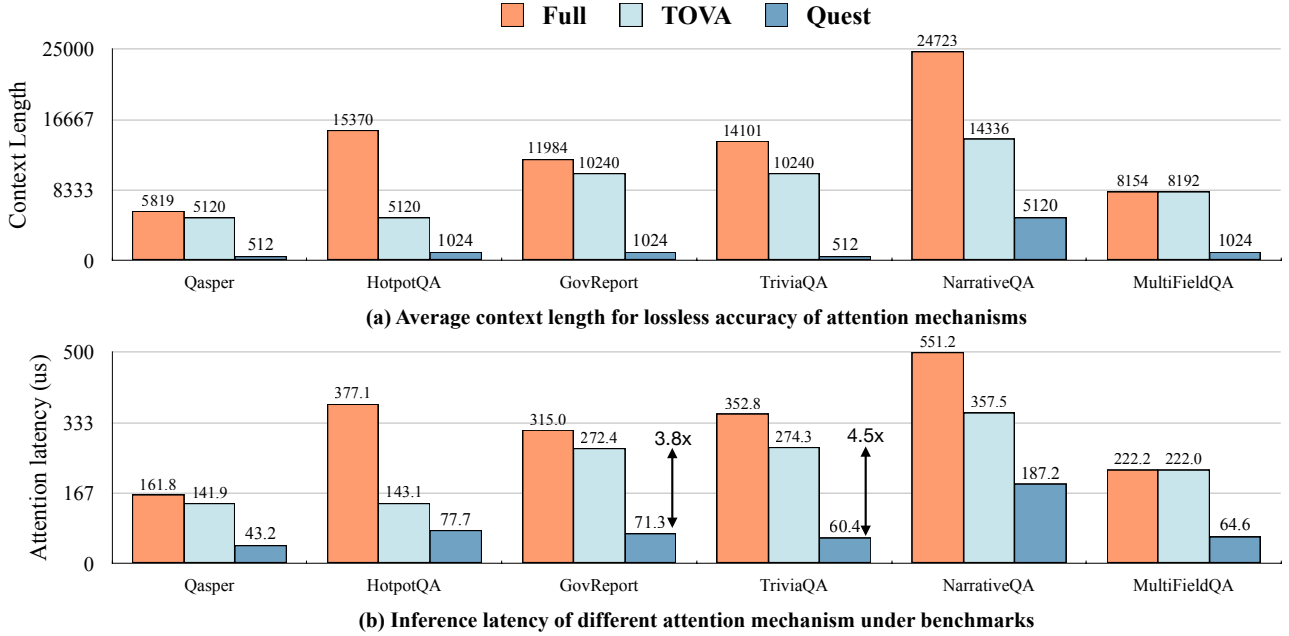


Figure 11. Efficiency comparison of Quest with baselines under the same accuracy constraint. (a) Tokens budgets needed for comparable accuracy by different attention methods. Full denotes the original attention, which means the average context length of benchmarks. (b) Inference latency of different attention methods for comparable accuracy. Quest boosts $3.82\times$ speed on GovReport compared to TOVA.

efficiency by utilizing the inference latency of FlashInfer, disregarding other runtime overheads (e.g., TOVA’s requirement to calculate history scores (Oren et al., 2024)). In contrast, Quest is evaluated in a practical setting with consideration of all operators. As shown in Fig. 11(b), Quest significantly surpasses all baselines in terms of self-attention latency due to the high query-aware sparsity. For GovReport and TriviaQA, Quest boosts the inference by $3.82\times$ and $4.54\times$, respectively. Therefore, Quest can achieve higher efficiency while maintaining superior accuracy.

5. Conclusion

We present Quest, an efficient and accurate KV cache selection algorithm that exploits query-aware sparsity. Quest dynamically estimates the criticality of tokens in KV cache based on the per-page metadata and the current query. It then performs self-attention only on the critical tokens with greatly reduced memory movement, providing high sparsity with negligible accuracy loss. Comprehensive evaluations demonstrate that Quest provides up to $7.03\times$ self-attention speedup, which contributes to $2.23\times$ end-to-end latency re-